

Credit Card Anomaly Detection Using Apache Kafka

Real-Time Transaction Monitoring, Anomaly Detection and
Alert Routing

Technical Project Documentation

Name: Harshita Pendli
Role: Data Engineering Trainee at Nexturn
Project: Credit Card Anomaly Detection Using Apache Kafka
Platform: Windows + PowerShell
Kafka: Apache Kafka 4.3.1 with KRaft
Language: Python
Storage: SQLite
Dashboard: Streamlit

September 2026

Contents

1	Project Overview	4
1.1	Main Objectives	4
2	Technologies and Environment	4
3	Architecture	5
3.1	Why Kafka	5
3.2	Why KRaft	5
3.3	Why SQLite	5
4	Kafka Configuration	6
4.1	Important Configuration Terms	6
5	Kafka Storage Formatting and Startup	7
5.1	Kafka Version Verification	7
6	Kafka Topics	8
7	Python Environment Setup	8
8	Project Directory Structure	9
9	Application Configuration	10
10	Transaction Generation	10
11	Kafka Producer	11
12	Kafka Consumer	12
12.1	Consumer Group	12
13	Transaction Validation	12
14	Dead-Letter Queue Pattern	13
15	Anomaly Detection Rules	13
15.1	Amount Anomaly	14
15.2	Location Anomaly	14
15.3	Velocity Anomaly	14
15.4	Frequency Anomaly	14
15.5	Time Anomaly	14
15.6	Duplicate Anomaly	15
16	Risk Scoring and Classification	15

17 Anomaly Detection Engine	15
18 Kafka Routing	16
19 SQLite Storage	17
20 Alert Service	17
21 JSON Outputs	18
22 Logging	18
23 Verification of High-Risk Detection	18
24 Verification of Normal Transaction	19
25 Verification of Suspicious Transaction	19
26 Statistics Verification	19
27 Streamlit Dashboard	20
28 Command Reference	21
28.1 Kafka Installation and Verification	21
28.2 KRaft Storage and Broker	21
28.3 Kafka Topic Commands	21
28.4 Python Environment	22
28.5 Source-Code Editing and Syntax Checks	22
28.6 Application Execution	23
28.7 Invalid Transaction Test	23
28.8 Output and Database Verification	23
28.9 Streamlit Dashboard	23
28.10Git Commands	24
29 Scope Decisions	24
30 Code Validation and Syntax Checking	24
31 Git and GitHub Version Control	25
32 Git Ignore Configuration	25
33 End-to-End Execution Procedure	26
34 Operational Notes	27
35 Final Verification	27

36 Screenshots and Execution Evidence	28
37 Conclusion	39

1 Project Overview

The project implements a real-time credit-card transaction processing and anomaly detection pipeline using Apache Kafka and Python. Transactions are generated continuously, published to Kafka, consumed by a detection engine, validated, analysed against multiple anomaly rules, classified according to a project-defined risk score, and routed to dedicated Kafka topics.

The processed information is also stored in SQLite. High-risk transactions generate alerts, JSON output files are maintained for reporting, application logs are generated for traceability, and a Streamlit dashboard provides a visual view of transaction and anomaly statistics.

The implementation was designed as a single-broker Kafka system so that the complete workflow could be developed and tested locally without the operational complexity of multiple brokers.

1.1 Main Objectives

- Generate realistic credit-card transactions continuously.
- Publish transaction events to Apache Kafka.
- Validate incoming JSON transaction records.
- Detect multiple types of transaction anomalies.
- Calculate a project-defined risk score.
- Classify transactions as NORMAL, SUSPICIOUS, or HIGH RISK.
- Route transactions to appropriate Kafka topics.
- Route invalid records to a dead-letter topic.
- Store processed transactions and alerts in SQLite.
- Generate JSON reports and processing statistics.
- Maintain application logs.
- Provide a Streamlit monitoring dashboard.

2 Technologies and Environment

Technology	Usage
Windows PowerShell	Command-line environment
Java 25.0.4	Runtime required by Apache Kafka
Apache Kafka 4.3.1	Event streaming and topic-based routing

KRaft	Kafka metadata management without ZooKeeper
Python	Application and anomaly detection logic
kafka-python 3.0.11	Python Kafka producer/consumer client
SQLite	Local transaction and alert persistence
JSON	Transaction and report interchange format
Streamlit	Monitoring dashboard
Git	Source-code version control
GitHub	Remote source-code repository

3 Architecture

The implemented flow is:

Transaction Generator → **Kafka Producer** → **credit-card-transactions** →
Anomaly Detection Engine

The detection engine performs validation and anomaly checks. Valid transactions are classified and routed as follows:

- NORMAL → normal-transactions
- SUSPICIOUS → anomaly-transactions
- HIGH RISK → high-risk-alerts
- INVALID → invalid-transactions

Valid processed transactions are stored in SQLite. High-risk transactions additionally generate alert records. JSON files and logs provide additional output and traceability.

3.1 Why Kafka

Apache Kafka is used because the project requires an event-streaming pipeline in which transaction events can be produced and consumed independently. Kafka topics provide a clean mechanism for separating normal, suspicious, high-risk, and invalid records.

3.2 Why KRaft

KRaft is Kafka's metadata management architecture that removes the dependency on ZooKeeper. The project uses a single Kafka node running both broker and controller roles.

3.3 Why SQLite

SQLite provides a lightweight local relational database for storing transactions and alerts without requiring a separate database server.

4 Kafka Configuration

Kafka was installed under:

```
C:\kafka
```

The project uses:

```
Kafka version: 4.3.1
Broker: localhost:9093
Controller: localhost:9193
Mode: KRaft
Deployment: Single broker
```

Port 9092 was already occupied by Docker/WSL services, so the Kafka client endpoint was configured on port 9093. The controller uses port 9193.

The relevant KRaft configuration is:

```
process.roles=broker,controller
node.id=1
controller.quorum.voters=1@localhost:9193
listeners=PLAINTEXT://:9093,CONTROLLER://:9193
advertised.listeners=PLAINTEXT://localhost:9093
controller.listener.names=CONTROLLER
inter.broker.listener.name=PLAINTEXT
listener.security.protocol.map=CONTROLLER:PLAINTEXT,PLAINTEXT:PLAINTEXT

num.network.threads=3
num.io.threads=8
socket.send.buffer.bytes=102400
socket.receive.buffer.bytes=102400
socket.request.max.bytes=104857600

log.dirs=C:/tmp/credit-card-kafka-logs
num.partitions=3
num.recovery.threads.per.data.dir=1

offsets.topic.replication.factor=1
transaction.state.log.replication.factor=1
transaction.state.log.min.isr=1
group.initial.rebalance.delay.ms=0
```

4.1 Important Configuration Terms

- **process.roles:** Defines the roles performed by the Kafka node.
- **node.id:** Unique identifier of the Kafka node.

- **controller.quorum.voters:** Defines the KRaft controller quorum.
- **listeners:** Network endpoints on which Kafka accepts connections.
- **advertised.listeners:** Endpoint advertised to Kafka clients.
- **log.dirs:** Location where Kafka stores its log data.
- **num.partitions:** Default number of partitions for created topics when not explicitly specified.
- **replication.factor:** Number of copies of a topic partition. The local project uses one broker, so replication factor is 1.

5 Kafka Storage Formatting and Startup

Before starting the KRaft broker, the Kafka storage directory was formatted.

```
cd C:\kafka

.\bin\windows\kafka-storage.bat format -t ZH_Nk6kIQ-mdAfI__bRVbw -c .\config\
server.properties
```

The command initialized the new KRaft storage directory:

```
C:\tmp\credit-card-kafka-logs
```

The Kafka broker was then started using:

```
.\bin\windows\kafka-server-start.bat .\config\server.properties
```

A successful startup displayed a message indicating that the Kafka server had started and was awaiting socket connections on port 9093.

5.1 Kafka Version Verification

```
C:\kafka\bin\windows\kafka-topics.bat --version
```

Expected version:

```
4.3.1
```

Some Kafka commands displayed a Log4j reconfiguration message. This did not prevent the commands or broker from functioning and was therefore treated as a non-blocking console message.

6 Kafka Topics

Five topics were created for the project:

Topic	Purpose
credit-card-transactions	Main input stream
normal-transactions	Valid transactions with no detected anomaly
anomaly-transactions	Suspicious transactions with one or more anomalies and score below 60
high-risk-alerts	High-risk transactions with score 60 or above
invalid-transactions	Invalid records routed to the dead-letter topic

Topic creation commands:

```
C:\kafka\bin\windows\kafka-topics.bat --create --topic credit-card-
transactions --bootstrap-server localhost:9093 --partitions 3 --replication
-factor 1

C:\kafka\bin\windows\kafka-topics.bat --create --topic normal-transactions --
bootstrap-server localhost:9093 --partitions 3 --replication-factor 1

C:\kafka\bin\windows\kafka-topics.bat --create --topic anomaly-transactions --
bootstrap-server localhost:9093 --partitions 3 --replication-factor 1

C:\kafka\bin\windows\kafka-topics.bat --create --topic high-risk-alerts --
bootstrap-server localhost:9093 --partitions 3 --replication-factor 1

C:\kafka\bin\windows\kafka-topics.bat --create --topic invalid-transactions --
bootstrap-server localhost:9093 --partitions 3 --replication-factor 1
```

Topic verification:

```
C:\kafka\bin\windows\kafka-topics.bat --list --bootstrap-server localhost:9093
```

The final topic list was:

```
anomaly-transactions
credit-card-transactions
high-risk-alerts
invalid-transactions
normal-transactions
```

7 Python Environment Setup

The project was created under:

```
C:\Users\harsh\credit-card-anomaly-detection
```

A Python virtual environment was created using:

```
python -m venv venv
```

The environment was activated in PowerShell:

```
.\venv\Scripts\Activate.ps1
```

The Kafka Python client was installed using:

```
pip install kafka-python==3.0.11
```

Streamlit was installed for the dashboard:

```
pip install streamlit
```

The project dependency file contains the main application dependencies:

```
Kafka-python==3.0.11
streamlit
pandas
```

8 Project Directory Structure

```
credit-card-anomaly-detection/
|
+-- producer/
| +-- transaction_generator.py
| +-- transaction_producer.py
|
+-- consumer/
| +-- transaction_consumer.py
| +-- anomaly_detector.py
|
+-- alerts/
| +-- alert_service.py
|
+-- storage/
| +-- database.py
|
+-- config/
| +-- config.py
|
+-- utils/
```

```
| +-- logger.py
| +-- validators.py
|
+-- data/
|
+-- output/
| +-- normal_transactions.json
| +-- anomaly_transactions.json
| +-- alerts.json
| +-- statistics.json
|
+-- logs/
| +-- anomaly_detector.log
|
+-- dashboard.py
+-- requirements.txt
+-- README.md
+-- .gitignore
```

The local virtual environment, SQLite database, generated output files, logs, cache files, and IDE-specific files were excluded from Git using `.gitignore`.

9 Application Configuration

The central Kafka configuration was kept in `config/config.py`.

```
KAFKA_BOOTSTRAP_SERVERS = "localhost:9093"

TRANSACTION_TOPIC = "credit-card-transactions"
NORMAL_TOPIC = "normal-transactions"
ANOMALY_TOPIC = "anomaly-transactions"
HIGH_RISK_TOPIC = "high-risk-alerts"
INVALID_TOPIC = "invalid-transactions"

CONSUMER_GROUP = "anomaly-detector-group"
```

Centralizing these values avoids repeatedly hard-coding Kafka connection details throughout the application.

10 Transaction Generation

The transaction generator creates records containing:

- Transaction ID
- Card ID

- Amount
- Location
- Merchant
- Transaction type
- Timestamp

The real-time producer intentionally generates mostly normal transactions while occasionally generating high-value transactions to make anomaly detection observable during testing.

The producer uses the Kafka topic:

```
credit-card-transactions
```

The producer was started with:

```
python producer\transaction_producer.py
```

The producer prints each generated transaction and continues until **Ctrl+C** is pressed.

11 Kafka Producer

The producer uses `KafkaProducer` from `kafka-python`.

Conceptually, the producer performs:

1. Generate a transaction.
2. Serialize the card ID as the Kafka message key.
3. Serialize the transaction dictionary as JSON.
4. Send the message to `credit-card-transactions`.
5. Flush the producer.
6. Display the generated transaction.
7. Wait briefly before generating the next transaction.

Example output:

```
Sent: TXN00001 | Card: CARD1002 | Amount: Rs.58163.81  
Sent: TXN00002 | Card: CARD1005 | Amount: Rs.3633.80  
Sent: TXN00003 | Card: CARD1002 | Amount: Rs.2701.03
```

12 Kafka Consumer

A separate consumer was implemented to demonstrate direct consumption from the input topic.

It was executed with:

```
python consumer\transaction_consumer.py
```

The consumer uses the consumer group:

```
anomaly-detector-group
```

The consumer reads JSON values and displays transaction ID, card ID, and amount.

12.1 Consumer Group

A Kafka consumer group allows consumers to coordinate the consumption of partitions. In this project, the anomaly detector uses one consumer group because a single detection process handles the local test workload.

13 Transaction Validation

Before anomaly analysis, every incoming record is validated.

Required fields are:

```
transaction_id
card_id
amount
location
merchant
transaction_type
timestamp
```

The validator checks:

- The message is a JSON object.
- Required fields exist.
- Transaction and card IDs are non-empty strings.
- Amount is numeric and greater than zero.
- Location and merchant are non-empty.
- Transaction type is present.
- Timestamp can be parsed as an ISO-formatted timestamp.

Invalid records are not allowed into the normal anomaly-analysis path.

14 Dead-Letter Queue Pattern

The topic `invalid-transactions` is used as a dead-letter topic.

A deliberately invalid record was sent without a timestamp and with a negative amount. The validator detected the missing timestamp first and returned:

```
Missing field: timestamp
```

The record was routed to:

```
invalid-transactions
```

The Kafka console consumer was used to verify the record:

```
C:\kafka\bin\windows\kafka-console-consumer.bat --bootstrap-server localhost
:9093 --topic invalid-transactions --from-beginning --max-messages 1
```

The output contained the original invalid transaction, the validation error, and the processing timestamp.

This demonstrates that malformed input is isolated instead of being allowed to stop the main processing pipeline.

15 Anomaly Detection Rules

The anomaly detector implements six detection categories plus duplicate detection.

Anomaly	Rule	Score
Amount	Amount greater than Rs.50,000	25
Location	Location changes within the configured 5-minute window	20
Velocity	More than 5 transactions within 60 seconds	20
Frequency	More than 10 transactions within one hour	15
Time	Transaction occurs between 00:00 and 05:00	10

Duplicate	Duplicate	transaction	25
	detected	within	30
	seconds		

These thresholds are **project-defined simulation rules**. They are not real banking or card-network fraud thresholds.

15.1 Amount Anomaly

A transaction is flagged when:

$$Amount > 50,000$$

For example:

```
Amount = Rs.58163.81
Threshold = Rs.50000
Result = Amount Anomaly
```

15.2 Location Anomaly

A transaction is flagged when the same card changes location within the configured location window.

Example:

```
Bangalore -> Mumbai within 6 seconds
Result = Location Anomaly
```

15.3 Velocity Anomaly

Velocity measures how many transactions occur within a short time interval.

$$Velocity > 5 \text{ transactions in } 60 \text{ seconds}$$

15.4 Frequency Anomaly

Frequency examines transaction activity over one hour.

$$Frequency > 10 \text{ transactions/hour}$$

15.5 Time Anomaly

Transactions occurring between midnight and 5:00 AM are flagged by the project rule.

15.6 Duplicate Anomaly

A transaction is checked against recently observed transactions to identify duplicate activity inside the configured 30-second window.

16 Risk Scoring and Classification

The risk score is the sum of the scores of detected anomaly rules, capped at 100.

$$RiskScore = \min(100, \sum AnomalyScores)$$

The project classification rules are:

Classification	Condition
NORMAL	No anomaly and score = 0
SUSPICIOUS	At least one anomaly and score < 60
HIGH RISK	Risk score $\geq$ 60

For example, a transaction with:

```
Amount Anomaly = 25
Location Anomaly = 20
Velocity Anomaly = 20
```

has:

$$25 + 20 + 20 = 65$$

Therefore, its project-defined classification is HIGH RISK.

17 Anomaly Detection Engine

The main detection engine is:

```
consumer\anomaly_detector.py
```

It performs the complete processing workflow:

1. Connect to Kafka.
2. Read a transaction from `credit-card-transactions`.
3. Validate the record.
4. Route invalid records to `invalid-transactions`.

5. Check amount anomaly.
6. Check location anomaly.
7. Check velocity anomaly.
8. Check frequency anomaly.
9. Check time anomaly.
10. Check duplicate activity.
11. Calculate risk score.
12. Determine risk level.
13. Save the transaction in SQLite.
14. Route the transaction to the appropriate Kafka topic.
15. Generate an alert when the transaction is high risk.
16. Update JSON output and statistics.
17. Write application logs.

The detector was started with:

```
python -m consumer.anomaly_detector
```

A successful startup displayed:

```
Kafka connected successfully.  
Listening to : credit-card-transactions  
Waiting for transactions...
```

18 Kafka Routing

The routing logic separates transaction outcomes:

```
NORMAL  
    -> normal-transactions  
  
SUSPICIOUS  
    -> anomaly-transactions  
  
HIGH RISK  
    -> high-risk-alerts  
  
INVALID  
    -> invalid-transactions
```

This separation makes it possible to consume, monitor, or process each category independently.

19 SQLite Storage

SQLite is used for persistent local storage.

The database file is:

```
credit_card_anomaly.db
```

The database contains two main tables:

1. transactions
2. alerts

The database initialization command was:

```
python storage/database.py
```

The successful output was:

```
SQLite database created successfully.  
Database: credit_card_anomaly.db
```

The transaction table stores transaction details, risk score, risk level, and anomaly descriptions.

The alert table stores alert ID, transaction ID, card ID, amount, risk score, risk level, alert type, anomaly list, message, and creation time.

20 Alert Service

High-risk transactions generate structured alerts.

The alert service creates an alert containing:

- Alert ID
- Transaction ID
- Card ID
- Amount
- Risk score
- Risk level
- Alert type

- Detected anomalies
- Alert message
- Creation timestamp

Example:

```
ALERT-TXN00015
Risk Score: 65
Risk Level: HIGH RISK
Anomalies:
Amount Anomaly
Location Anomaly
Velocity Anomaly
```

21 JSON Outputs

The application generates JSON outputs under the `output` directory.

File	Purpose
<code>normal_transactions.json</code>	Normal transaction records
<code>anomaly_transactions.json</code>	Suspicious and high-risk anomaly records
<code>alerts.json</code>	Generated high-risk alerts
<code>statistics.json</code>	Processing and anomaly statistics

22 Logging

Application logging is stored in:

```
logs\anomaly_detector.log
```

The logger records processing information, warnings for invalid records, high-risk alerts, and transaction save events.

The log format includes timestamp, log level, and message.

23 Verification of High-Risk Detection

A high-risk transaction was successfully detected:

```
Transaction ID : TXN00015
Card ID : CARD1005
Amount : Rs.88998.91
Risk Score : 65
Risk Level : HIGH RISK
```

```
Anomalies : Amount Anomaly,  
            Location Anomaly,  
            Velocity Anomaly
```

The detector displayed:

```
ALERT ROUTED -> high-risk-alerts  
HIGH RISK ALERT | TXN00015 | Score=65
```

The corresponding alert was also stored in SQLite and written to `alerts.json`.

24 Verification of Normal Transaction

A normal transaction was successfully processed:

```
Transaction ID : TXN00003  
Card ID : CARD1002  
Risk Score : 0  
Risk Level : NORMAL  
Anomalies : None
```

The transaction was routed to:

```
normal-transactions
```

25 Verification of Suspicious Transaction

A suspicious transaction was also observed:

```
Transaction ID : TXN00010  
Card ID : CARD1002  
Risk Score : 40  
Risk Level : SUSPICIOUS  
Anomalies : Location Anomaly,  
            Velocity Anomaly
```

The transaction was routed to:

```
anomaly-transactions
```

26 Statistics Verification

One verified statistics output contained:

```
total_transactions : 17
normal_transactions : 6
suspicious_transactions : 10
high_risk_transactions : 1
invalid_transactions : 0
amount_anomalies : 5
location_anomalies : 8
velocity_anomalies : 2
frequency_anomalies : 0
time_anomalies : 0
duplicate_anomalies : 0
alerts_generated : 1
transactions_saved : 16
processing_errors : 0
```

The statistics represent a test run and are not intended as permanent production metrics.

27 Streamlit Dashboard

A Streamlit dashboard was added after the Kafka processing pipeline was completed.

The dashboard is implemented in:

```
dashboard.py
```

It reads data from SQLite and the generated statistics and provides:

- Transaction overview
- Normal transaction count
- Suspicious transaction count
- High-risk transaction count
- Invalid transaction count
- Alert summary
- Processing error count
- Risk distribution chart
- Anomaly statistics chart
- High-risk alert records
- Recent transaction records
- Risk-level filtering

- Dashboard refresh

The dashboard was started using:

```
streamlit run dashboard.py
```

It was accessed locally at:

```
http://localhost:8501
```

The verified dashboard displayed:

```
Total Transactions : 172
Normal : 7
Suspicious : 132
High Risk : 33
Invalid : 2
Alerts Generated : 33
Transactions Saved : 172
Processing Errors : 0
```

28 Command Reference

This section collects the principal commands used during setup, development, testing, verification, dashboard execution, and Git version control.

28.1 Kafka Installation and Verification

```
cd C:\kafka
.\bin\windows\kafka-topics.bat --version
```

28.2 KRaft Storage and Broker

```
.\bin\windows\kafka-storage.bat format -t ZH_Nk6kIQ-mdAfI__bRVbw -c .\config\
server.properties

.\bin\windows\kafka-server-start.bat .\config\server.properties
```

28.3 Kafka Topic Commands

```
C:\kafka\bin\windows\kafka-topics.bat --list --bootstrap-server localhost:9093

C:\kafka\bin\windows\kafka-topics.bat --create --topic credit-card-
transactions --bootstrap-server localhost:9093 --partitions 3 --replication
-factor 1
```

```
C:\kafka\bin\windows\kafka-topics.bat --create --topic normal-transactions --
bootstrap-server localhost:9093 --partitions 3 --replication-factor 1

C:\kafka\bin\windows\kafka-topics.bat --create --topic anomaly-transactions --
bootstrap-server localhost:9093 --partitions 3 --replication-factor 1

C:\kafka\bin\windows\kafka-topics.bat --create --topic high-risk-alerts --
bootstrap-server localhost:9093 --partitions 3 --replication-factor 1

C:\kafka\bin\windows\kafka-topics.bat --create --topic invalid-transactions --
bootstrap-server localhost:9093 --partitions 3 --replication-factor 1
```

28.4 Python Environment

```
cd C:\Users\harsh\credit-card-anomaly-detection
python -m venv venv
.\venv\Scripts\Activate.ps1

pip install kafka-python==3.0.11
pip install streamlit
```

28.5 Source-Code Editing and Syntax Checks

The project files were opened during development using commands such as:

```
notepad alerts\alert_service.py
notepad consumer\anomaly_detector.py
notepad config\config.py
```

Main syntax verification:

```
python -m py_compile consumer\anomaly_detector.py utils\validators.py utils\
logger.py alerts\alert_service.py storage\database.py

python -m py_compile dashboard.py
```

Database verification commands used during debugging included:

```
python -m py_compile storage\database.py

python -c "from storage.database import save_alert; print('save_alert imported
successfully')"
```

```
python storage\database.py
```

28.6 Application Execution

```
python -m consumer.anomaly_detector

python producer\transaction_producer.py

python consumer\transaction_consumer.py
```

28.7 Invalid Transaction Test

A deliberately invalid message was published to verify validation and dead-letter routing. The test command used the project configuration and omitted the timestamp:

```
python -c "import sys,os,json; sys.path.append(os.getcwd()); from kafka import
    KafkaProducer; from config.config import KAFKA_BOOTSTRAP_SERVERS,
    TRANSACTION_TOPIC; p=KafkaProducer(bootstrap_servers=
    KAFKA_BOOTSTRAP_SERVERS,value_serializer=lambda v: json.dumps(v).encode('
    utf-8')); p.send(TRANSACTION_TOPIC,value={'transaction_id':'INVALID001','
    card_id':'CARD9999','amount':-500,'location':'Hyderabad','merchant':'
    TestMerchant','transaction_type':'PURCHASE'}); p.flush(); p.close(); print
    ('Invalid transaction sent')"
```

The dead-letter topic was then inspected with:

```
C:\kafka\bin\windows\kafka-console-consumer.bat --bootstrap-server localhost
:9093 --topic invalid-transactions --from-beginning --max-messages 1
```

28.8 Output and Database Verification

```
Get-Content output\statistics.json

Get-Content output>alerts.json

Get-ChildItem output, logs

python -c "from storage.database import get_high_risk_alerts; print(
    get_high_risk_alerts())"
```

28.9 Streamlit Dashboard

```
streamlit run dashboard.py
```

The local dashboard was opened at:

```
http://localhost:8501
```

28.10 Git Commands

```
git init
git rev-parse --show-toplevel
git status
git add .
git commit -m "Initial project implementation and documentation"

git remote add origin https://github.com/Harshita245-tech/credit-card-anomaly-
    detection-kafka.git
git remote -v
git push -u origin master
```

29 Scope Decisions

The project focuses on the implemented Kafka streaming, anomaly detection, validation, routing, SQLite storage, alerting, JSON outputs, logging, dashboard, and GitHub workflow.

The following optional items were intentionally left out of the final implementation:

- A separate sample CSV dataset.
- A unit-test suite.

These omissions do not change the implemented end-to-end Kafka processing workflow demonstrated in this documentation.

30 Code Validation and Syntax Checking

Python syntax checks were performed before execution.

For the main modules:

```
python -m py_compile consumer\anomaly_detector.py utils\validators.py utils\
    logger.py alerts\alert_service.py storage\database.py
```

A successful `py_compile` command returns to the PowerShell prompt without an error message.

The dashboard was also checked using:

```
python -m py_compile dashboard.py
```

No output indicated that the dashboard Python file passed the syntax check.

31 Git and GitHub Version Control

Git was used to maintain the project source code.

Inside the project directory:

```
git init
```

The repository location was verified with:

```
git rev-parse --show-toplevel
```

Project files were staged with:

```
git add .
```

Repository status was checked using:

```
git status
```

The initial project commit was created using:

```
git commit -m "Initial project implementation and documentation"
```

The GitHub remote repository was:

```
https://github.com/Harshita245-tech/credit-card-anomaly-detection-kafka.git
```

The remote was configured using:

```
git remote add origin https://github.com/Harshita245-tech/credit-card-anomaly-detection-kafka.git
```

The remote was verified using:

```
git remote -v
```

The final source was pushed using:

```
git push -u origin master
```

32 Git Ignore Configuration

The following files and directories were excluded from version control:

```
venv/  
__pycache__/  
*.pyc  
*.db
```

```
logs/  
output/  
.env  
.vscode/  
.idea/
```

This prevents local runtime data, virtual-environment files, generated logs, SQLite databases, and generated outputs from being committed unnecessarily.

33 End-to-End Execution Procedure

The complete workflow can be reproduced in the following order.

Step 1: Activate the Python Environment

```
cd C:\Users\harsh\credit-card-anomaly-detection  
.\venv\Scripts\Activate.ps1
```

Step 2: Start Kafka

In the Kafka terminal:

```
cd C:\kafka  
.\bin\windows\kafka-server-start.bat .\config\server.properties
```

Step 3: Start the Anomaly Detector

In the project terminal:

```
python -m consumer.anomaly_detector
```

Step 4: Start the Producer

In another project terminal:

```
python producer\transaction_producer.py
```

Step 5: Observe Processing

The detector validates each record, checks anomalies, calculates the risk score, saves the transaction, and routes it to the corresponding topic.

Step 6: Start the Dashboard

```
streamlit run dashboard.py
```

Then open:

```
http://localhost:8501
```

34 Operational Notes

The final setup intentionally uses a single Kafka broker. This keeps the project easy to run locally while still demonstrating the Kafka event-streaming architecture.

The main runtime components are:

1. Kafka broker terminal
2. Anomaly detector terminal
3. Producer terminal
4. Streamlit dashboard terminal

Kafka configuration was kept stable after the broker was successfully running on port 9093.

The Kafka console warnings related to Log4j and KIP-848 did not prevent successful processing and were not treated as application failures.

35 Final Verification

The completed implementation demonstrated all of the following:

- Kafka broker successfully running with KRaft.
- Five Kafka topics successfully created.
- Real-time producer successfully publishing transactions.
- Consumer successfully receiving transactions.
- Transaction validation successfully implemented.
- Invalid transaction successfully routed to the dead-letter topic.
- Amount anomaly successfully detected.
- Location anomaly successfully detected.
- Velocity anomaly successfully detected.
- Frequency anomaly successfully detected.

- Time anomaly rule implemented.
- Duplicate anomaly rule implemented.
- Risk scoring successfully implemented.
- NORMAL, SUSPICIOUS, and HIGH RISK classification successfully demonstrated.
- High-risk alert successfully generated.
- SQLite transaction storage successfully verified.
- SQLite alert storage successfully verified.
- JSON outputs successfully generated.
- Statistics successfully generated.
- Application logging successfully generated.
- Streamlit dashboard successfully launched.
- Dashboard transaction and risk information successfully displayed.
- Git repository successfully initialized and source files staged.
- Project source code pushed to GitHub.

36 Screenshots and Execution Evidence

The screenshots below are included in the exact order in which they were provided. The original uploaded filenames are preserved.

```
(venv) PS C:\Users\harsh\credit-card-anomaly-detection> git add .
(venv) PS C:\Users\harsh\credit-card-anomaly-detection> git status
On branch master

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
    new file:   .gitignore
    new file:   README.md
    new file:   alerts/__init__.py
    new file:   alerts/alert_service.py
    new file:   config/__init__.py
    new file:   config/config.py
    new file:   consumer/anomaly_detector.py
    new file:   consumer/transaction_consumer.py
    new file:   producer/__init__.py
    new file:   producer/transaction_generator.py
    new file:   producer/transaction_producer.py
    new file:   requirements.txt
    new file:   storage/database.py
    new file:   utils/__init__.py
    new file:   utils/logger.py
    new file:   utils/validators.py
```

Figure 1: Git staging verification showing the project files added to the Git repository.

```
(venv) PS C:\Users\harsh\credit-card-anomaly-detection> python -c "import sys
,os,json; sys.path.append(os.getcwd()); from kafka import KafkaProducer; from
config.config import KAFKA_BOOTSTRAP_SERVERS,TRANSACTION_TOPIC; p=KafkaProdu
cer(bootstrap_servers=KAFKA_BOOTSTRAP_SERVERS,value_serializer=lambda v: json
.dumps(v).encode('utf-8')); p.send(TRANSACTION_TOPIC,value={'transaction_id':
'INVALID001','card_id':'CARD9999','amount':-500,'location':'Hyderabad','merch
ant':'TestMerchant','transaction_type':'PURCHASE'}); p.flush(); p.close(); pr
int('Invalid transaction sent')
Invalid transaction sent
(venv) PS C:\Users\harsh\credit-card-anomaly-detection> C:\kafka\bin\windows\
kafka-console-consumer.bat --bootstrap-server localhost:9093 --topic invalid-
transactions --from-beginning --max-messages 1
2026-09-22T04:53:37.654534800Z main ERROR Reconfiguration failed: No configur
ation found for '764c12b6' at 'null' in 'null'
The consumer rebalance protocol (KIP-848) is production-ready! Set group.prot
ocol=consumer to try it out. See https://kafka.apache.org/documentation/#cons
umer_rebalance_protocol
{"transaction": {"transaction_id": "INVALID001", "card_id": "CARD9999", "amou
nt": -500, "location": "Hyderabad", "merchant": "TestMerchant", "transaction_
type": "PURCHASE"}, "error": "Missing field: timestamp", "timestamp": "2026-0
9-22T10:20:40.615941"}
Processed a total of 1 messages
(venv) PS C:\Users\harsh\credit-card-anomaly-detection>
```

Figure 2: Manual invalid transaction sent to Kafka and verification of the invalid-transactions dead-letter topic.

```

=====
Processing: TXN00178 | Card: CARD1002 | Amount: ₹4067.46
2026-09-22 10:20:40,562 | INFO | Processing: TXN00178 | Card: CARD1002 | Amount: ₹4067.46
✓ Amount normal | ₹4067.46
⚠ LOCATION ANOMALY | Bangalore → Mumbai within 6 seconds
⚠ VELOCITY ANOMALY | 6 transactions within 60 seconds
⚠ FREQUENCY ANOMALY | 34 transactions in last hour
✓ Transaction time normal | 10:18:49
✓ Duplicate check normal
2026-09-22 10:20:40,573 | INFO | Transaction saved: TXN00178
📦 ROUTED → anomaly-transactions

***** TRANSACTION RESULT *****
Transaction ID : TXN00178
Card ID       : CARD1002
Risk Score    : 55
Risk Level    : SUSPICIOUS
Anomalies     : Location Anomaly, Velocity Anomaly, Frequency Anomaly
*****

=====
❌ INVALID TRANSACTION | INVALID001
Reason: Missing field: timestamp
📦 ROUTED → invalid-transactions
=====

2026-09-22 10:20:40,614 | WARNING | Invalid transaction: INVALID001 | Missing field: timestamp

=====
❌ INVALID TRANSACTION | INVALID001
Reason: Missing field: timestamp
📦 ROUTED → invalid-transactions
=====

2026-09-22 10:21:49,343 | WARNING | Invalid transaction: INVALID001 | Missing field: timestamp

```

Figure 3: Anomaly detector processing a suspicious transaction and routing an invalid transaction to the invalid-transactions topic.

```

=====
Processing: TXN00171 | Card: CARD1002 | Amount: ₹98799.60
2026-09-22 10:20:40,425 | INFO | Processing: TXN00171 | Card: CARD1002 | Amount: ₹98799.60
⚠️ AMOUNT ANOMALY | ₹98799.60 > ₹50000
⚠️ LOCATION ANOMALY | Hyderabad → Delhi within 2 seconds
⚠️ VELOCITY ANOMALY | 7 transactions within 60 seconds
⚠️ FREQUENCY ANOMALY | 32 transactions in last hour
✓ Transaction time normal | 10:18:34
✓ Duplicate check normal
2026-09-22 10:20:40,437 | INFO | Transaction saved: TXN00171
🔥 ALERT ROUTED → high-risk-alerts
2026-09-22 10:20:40,486 | WARNING | HIGH RISK ALERT | TXN00171 | Score=80 | Anomalies=Amount Anomaly, Location Anomaly, Velocity Anomaly, Frequency Anomaly

***** TRANSACTION RESULT *****
Transaction ID : TXN00171
Card ID       : CARD1002
Risk Score    : 80
Risk Level    : HIGH RISK
Anomalies     : Amount Anomaly, Location Anomaly, Velocity Anomaly, Frequency Anomaly
*****

```

Figure 4: High-risk transaction detection showing amount, location, velocity, and frequency anomalies with a risk score of 80.

```

(venv) PS C:\Users\harsh\credit-card-anomaly-detection> Get-Content output\statistics.json
{
  "total_transactions": 17,
  "normal_transactions": 6,
  "suspicious_transactions": 10,
  "high_risk_transactions": 1,
  "invalid_transactions": 0,
  "amount_anomalies": 5,
  "location_anomalies": 8,
  "velocity_anomalies": 2,
  "frequency_anomalies": 0,
  "time_anomalies": 0,
  "duplicate_anomalies": 0,
  "alerts_generated": 1,
  "transactions_saved": 16,
  "processing_errors": 0
}
(venv) PS C:\Users\harsh\credit-card-anomaly-detection>

```

Figure 5: Statistics JSON verification showing transaction, anomaly, alert, and processing statistics.

```

(venv) PS C:\Users\harsh\credit-card-anomaly-detection> python -c "from storage.database import get_high_risk_alerts; print(get_high_risk_alerts())"
[('ALERT-TXN00015', 'TXN00015', 'CARD1005', 88998.91, 65, 'Amount Anomaly, Location Anomaly, Velocity Anomaly', '2026-09-22T10:13:22.355986')]
(venv) PS C:\Users\harsh\credit-card-anomaly-detection>

```

Figure 6: SQLite verification of the stored high-risk alert record.

```
(venv) PS C:\Users\harsh\credit-card-anomaly-detection> Get-Content output\alerts.json
[
  {
    "alert_id": "ALERT-TXN00015",
    "transaction_id": "TXN00015",
    "card_id": "CARD1005",
    "amount": 88998.91,
    "risk_score": 65,
    "risk_level": "HIGH RISK",
    "alert_type": "CREDIT_CARD_ANOMALY",
    "anomalies": [
      "Amount Anomaly",
      "Location Anomaly",
      "Velocity Anomaly"
    ],
    "message": "High-risk credit card transaction detected",
    "created_at": "2026-09-22T10:13:22.355986"
  }
]
(venv) PS C:\Users\harsh\credit-card-anomaly-detection>
```

Figure 7: Verification of the generated alerts.json file.

```
=====
Processing: TXN00015 | Card: CARD1005 | Amount: ₹88998.91
2026-09-22 10:13:22,309 | INFO | Processing: TXN00015 | Card: CARD1005 | Amount: ₹88998.91
⚠️ AMOUNT ANOMALY | ₹88998.91 > ₹50000
⚠️ LOCATION ANOMALY | Hyderabad → Mumbai within 14 seconds
⚠️ VELOCITY ANOMALY | 5 transactions within 60 seconds
✓ Frequency normal | 5 transaction(s) in last hour
✓ Transaction time normal | 10:13:22
✓ Duplicate check normal
2026-09-22 10:13:22,325 | INFO | Transaction saved: TXN00015
🚨 ALERT ROUTED → high-risk-alerts
2026-09-22 10:13:22,368 | WARNING | HIGH RISK ALERT | TXN00015 | Score=65 | Anomalies=Amount Anomaly, Location Anomaly, Velocity Anomaly

***** TRANSACTION RESULT *****
Transaction ID : TXN00015
Card ID       : CARD1005
Risk Score    : 65
Risk Level    : HIGH RISK
Anomalies     : Amount Anomaly, Location Anomaly, Velocity Anomaly
*****
```

Figure 8: High-risk transaction TXN00015 detected and routed to the high-risk-alerts topic.

```
(venv) PS C:\Users\harsh\credit-card-anomaly-detection> Get-ChildItem output
, logs

Directory: C:\Users\harsh\credit-card-anomaly-detection\output

Mode                LastWriteTime         Length Name
----                -
-a-----         21-09-2026         18:57         2395 anomaly_transactions.json
-a-----         21-09-2026         18:57         2447 normal_transactions.json
-a-----         21-09-2026         18:57          430 statistics.json

Directory: C:\Users\harsh\credit-card-anomaly-detection\logs

Mode                LastWriteTime         Length Name
----                -
-a-----         21-09-2026         18:57         2236 anomaly_detector.log
```

Figure 9: Verification of generated output JSON files and the anomaly detector log file.

```
=====
Processing: TXN00010 | Card: CARD1002 | Amount: ₹5487.38
2026-09-21 18:57:14,520 | INFO | Processing: TXN00010 | Card: CARD1002 | Amo
unt: ₹5487.38
✓ Amount normal | ₹5487.38
⚠ LOCATION ANOMALY | Hyderabad → Delhi within 8 seconds
⚠ VELOCITY ANOMALY | 5 transactions within 60 seconds
✓ Frequency normal | 5 transaction(s) in last hour
✓ Transaction time normal | 18:57:14
✓ Duplicate check normal
2026-09-21 18:57:14,533 | INFO | Transaction saved: TXN00010
📦 ROUTED → anomaly-transactions

***** TRANSACTION RESULT *****
Transaction ID : TXN00010
Card ID       : CARD1002
Risk Score    : 40
Risk Level    : SUSPICIOUS
Anomalies     : Location Anomaly, Velocity Anomaly
*****
```

Figure 10: Suspicious transaction processing with location and velocity anomalies.

```

=====
Processing: TXN00003 | Card: CARD1002 | Amount: ₹2701.03
2026-09-21 18:57:00,475 | INFO | Processing: TXN00003 | Card: CARD1002 | Amount: ₹2701.03
✓ Amount normal | ₹2701.03
✓ Location normal | Hyderabad
✓ Velocity normal | 2 transaction(s) in last minute
✓ Frequency normal | 2 transaction(s) in last hour
✓ Transaction time normal | 18:57:00
✓ Duplicate check normal
2026-09-21 18:57:00,489 | INFO | Transaction saved: TXN00003
📦 ROUTED → normal-transactions

***** TRANSACTION RESULT *****
Transaction ID : TXN00003
Card ID       : CARD1002
Risk Score    : 0
Risk Level    : NORMAL
Anomalies     : None
*****

```

Figure 11: Normal transaction processing with risk score 0 and no anomalies.

```

=====
Processing: TXN00001 | Card: CARD1002 | Amount: ₹58163.81
2026-09-21 18:56:56,464 | INFO | Processing: TXN00001 | Card: CARD1002 | Amount: ₹58163.81
⚠️ AMOUNT ANOMALY | ₹58163.81 > ₹50000
✓ Location normal | Hyderabad
✓ Velocity normal | 1 transaction(s) in last minute
✓ Frequency normal | 1 transaction(s) in last hour
✓ Transaction time normal | 18:56:56
✓ Duplicate check normal
2026-09-21 18:56:56,470 | INFO | Transaction saved: TXN00001
📦 ROUTED → anomaly-transactions

***** TRANSACTION RESULT *****
Transaction ID : TXN00001
Card ID       : CARD1002
Risk Score    : 25
Risk Level    : SUSPICIOUS
Anomalies     : Amount Anomaly
*****

```

Figure 12: Amount anomaly detection for a high-value transaction.

```
(venv) PS C:\Users\harsh\credit-card-anomaly-detection> python producer\transaction_producer.py
=====
REAL-TIME CREDIT CARD PRODUCER
=====
Sending transactions to Kafka...
Press Ctrl+C to stop.

Sent: TXN00001 | Card: CARD1002 | Amount: ₹58163.81
Sent: TXN00002 | Card: CARD1005 | Amount: ₹3633.80
Sent: TXN00003 | Card: CARD1002 | Amount: ₹2701.03
Sent: TXN00004 | Card: CARD1002 | Amount: ₹3671.43
Sent: TXN00005 | Card: CARD1005 | Amount: ₹939.40
Sent: TXN00006 | Card: CARD1002 | Amount: ₹6411.76
Sent: TXN00007 | Card: CARD1004 | Amount: ₹2216.75
Sent: TXN00008 | Card: CARD1003 | Amount: ₹6701.52
Sent: TXN00009 | Card: CARD1005 | Amount: ₹7309.95
Sent: TXN00010 | Card: CARD1002 | Amount: ₹5487.38
Sent: TXN00011 | Card: CARD1005 | Amount: ₹136.85
Sent: TXN00012 | Card: CARD1001 | Amount: ₹4058.93
Sent: TXN00013 | Card: CARD1001 | Amount: ₹2143.68

Producer stopped.
```

Figure 13: Real-time producer generating and sending transaction records to Kafka.

```

alerts\alert_service.py
(venv) PS C:\Users\harsh\credit-card-anomaly-detection> python -c "import co
nsumer.anomaly_detector"
2026-09-21 18:40:28,619 | INFO | Credit Card Anomaly Detection Engine starte
d
<string>:1: DeprecationWarning: key_serializer does not implement kafka.seri
alizer.Serializer
<string>:1: DeprecationWarning: value_serializer does not implement kafka.se
rializer.Serializer

=====
      CREDIT CARD ANOMALY DETECTOR
=====

Kafka connected successfully.
Listening to : credit-card-transactions

Detection Rules:
Amount threshold      : ₹50000
Velocity limit        : 5 transactions / 60 seconds
Frequency limit       : 10 transactions / hour
Location window       : 300 seconds
Duplicate window      : 30 seconds

Kafka routing enabled.
Input validation enabled.
SQLite storage enabled.
Alert database enabled.
JSON output enabled.
Statistics enabled.
Application logging enabled.

Waiting for transactions...

```

Figure 14: Anomaly detector startup showing Kafka connection, detection rules, routing, SQLite, JSON, statistics, and logging configuration.

```

(venv) PS C:\Users\harsh\credit-card-anomaly-detection> notepad alerts\alert_service.py
(venv) PS C:\Users\harsh\credit-card-anomaly-detection> notepad consumer\anomaly_detector.py
(venv) PS C:\Users\harsh\credit-card-anomaly-detection> notepad config\config.py
(venv) PS C:\Users\harsh\credit-card-anomaly-detection> python -m py_compile consumer\anomaly_detector.py utils\validators.py utils\logger.py alerts\alert_s
ervice.py storage\database.py
(venv) PS C:\Users\harsh\credit-card-anomaly-detection> python storage\database.py
SQLite database created successfully.
Database: credit_card_anomaly.db
(venv) PS C:\Users\harsh\credit-card-anomaly-detection> |

```

Figure 15: Project setup and source-code/configuration verification.

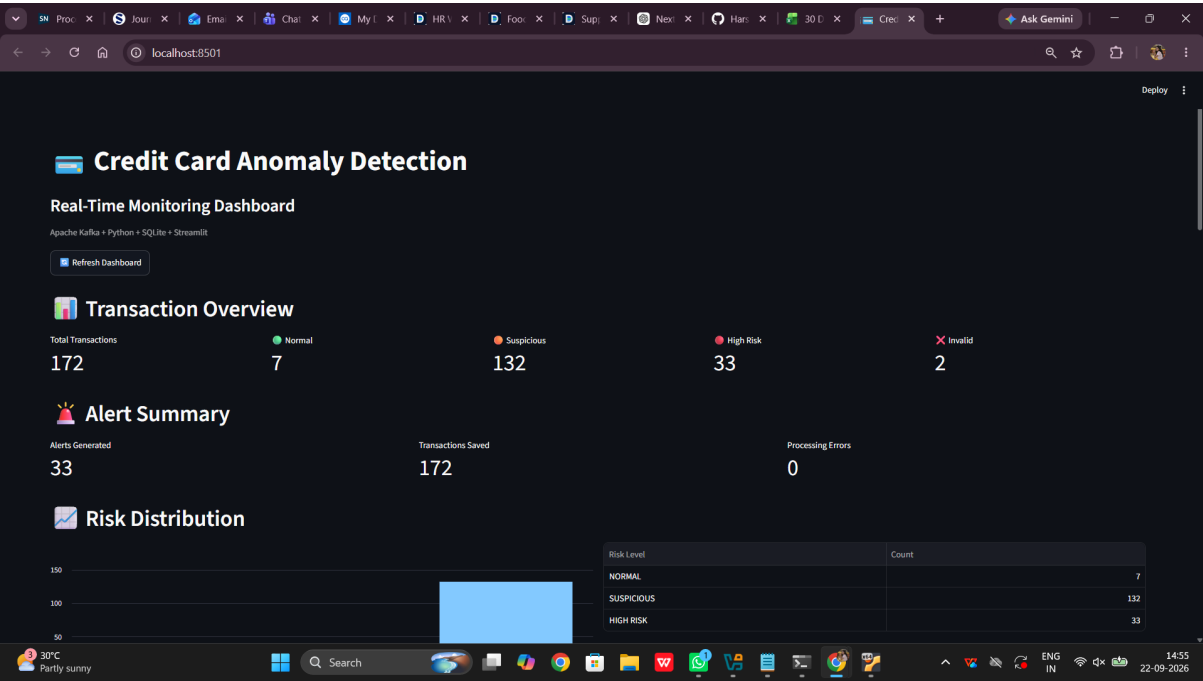


Figure 16: Complete Streamlit Credit Card Anomaly Detection dashboard view.



Figure 17: Streamlit dashboard transaction overview and alert summary.

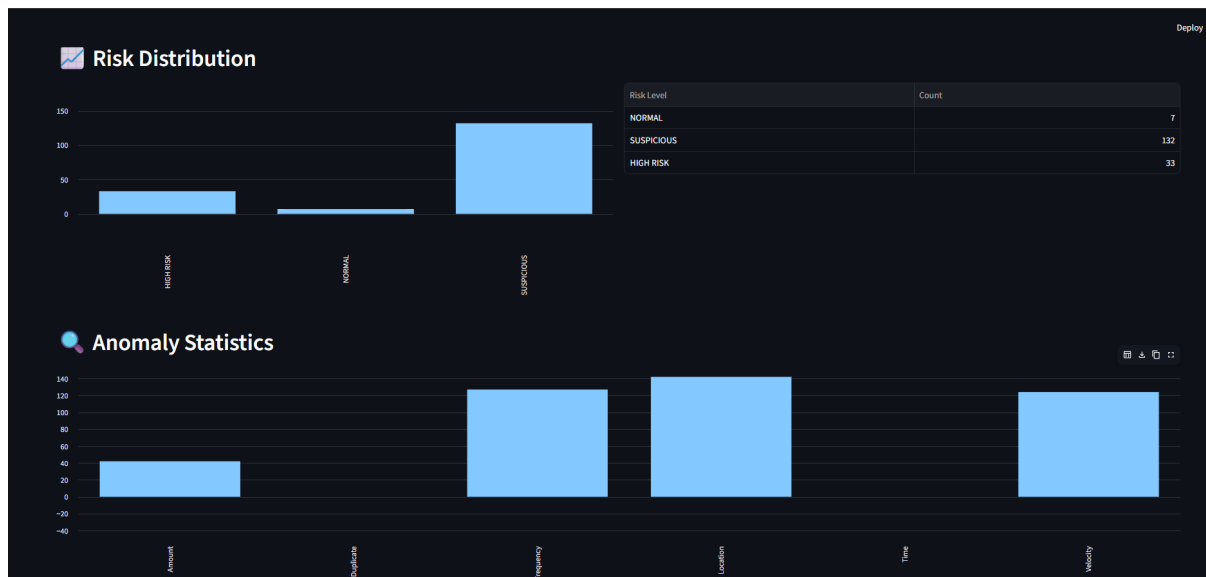


Figure 18: Streamlit dashboard risk distribution and anomaly statistics charts.

transaction_id	card_id	amount	location	merchant	transaction_type	timestamp	risk_score	risk_level	anomalies
TXN00137	CARD1002	2140.18	Mumbai	Uber	PURCHASE	2026-09-22T10:17:26	35	SUSPICIOUS	Location Anomaly, Frequency Anomaly
TXN00131	CARD1002	4407.43	Delhi	Amazon	PURCHASE	2026-09-22T10:17:14	35	SUSPICIOUS	Location Anomaly, Frequency Anomaly
TXN00106	CARD1002	5429.11	Chennai	Flipkart	PURCHASE	2026-09-22T10:16:24	35	SUSPICIOUS	Location Anomaly, Frequency Anomaly
TXN00103	CARD1002	81586.95	Delhi	Amazon	PURCHASE	2026-09-22T10:16:18	40	SUSPICIOUS	Amount Anomaly, Frequency Anomaly
TXN00092	CARD1002	93573.14	Delhi	Amazon	PURCHASE	2026-09-22T10:15:56	80	HIGH RISK	Amount Anomaly, Location Anomaly, Velocity Anomaly, Frequency Anomaly
TXN00083	CARD1002	3388.52	Bangalore	Uber	PURCHASE	2026-09-22T10:15:38	55	SUSPICIOUS	Location Anomaly, Velocity Anomaly, Frequency Anomaly
TXN00072	CARD1002	61792.94	Hyderabad	Restaurant	PURCHASE	2026-09-22T10:15:16	80	HIGH RISK	Amount Anomaly, Location Anomaly, Velocity Anomaly, Frequency Anomaly
TXN00064	CARD1002	84342.54	Mumbai	Uber	PURCHASE	2026-09-22T10:15:00	80	HIGH RISK	Amount Anomaly, Location Anomaly, Velocity Anomaly, Frequency Anomaly
TXN00062	CARD1002	1199.79	Bangalore	Walmart	PURCHASE	2026-09-22T10:14:56	55	SUSPICIOUS	Location Anomaly, Velocity Anomaly, Frequency Anomaly
TXN00057	CARD1002	7298.55	Hyderabad	Walmart	PURCHASE	2026-09-22T10:14:46	55	SUSPICIOUS	Location Anomaly, Velocity Anomaly, Frequency Anomaly

Figure 19: Streamlit dashboard recent transactions view with risk scores and anomaly descriptions.

transaction_id	card_id	amount	location	merchant	transaction_type	timestamp	risk_score	risk_level	anomalies
TXN00178	CARD1002	4067.46	Mumbai	Amazon	PURCHASE	2026-09-22T10:18:49	55	SUSPICIOUS	Location Anomaly, Velocity Anomaly, Frequency Anomaly
TXN00175	CARD1002	59509.52	Bangalore	Restaurant	PURCHASE	2026-09-22T10:18:43	80	HIGH RISK	Amount Anomaly, Location Anomaly, Velocity Anomaly, Frequency Anomaly
TXN00171	CARD1002	98799.6	Delhi	Flipkart	PURCHASE	2026-09-22T10:18:34	80	HIGH RISK	Amount Anomaly, Location Anomaly, Velocity Anomaly, Frequency Anomaly
TXN00170	CARD1002	65150.83	Hyderabad	Amazon	PURCHASE	2026-09-22T10:18:32	80	HIGH RISK	Amount Anomaly, Location Anomaly, Velocity Anomaly, Frequency Anomaly
TXN00164	CARD1002	5054.74	Bangalore	Restaurant	PURCHASE	2026-09-22T10:18:20	55	SUSPICIOUS	Location Anomaly, Velocity Anomaly, Frequency Anomaly
TXN00158	CARD1002	1097.99	Chennai	Flipkart	PURCHASE	2026-09-22T10:18:08	55	SUSPICIOUS	Location Anomaly, Velocity Anomaly, Frequency Anomaly
TXN00146	CARD1002	1139.56	Bangalore	Walmart	PURCHASE	2026-09-22T10:17:44	35	SUSPICIOUS	Velocity Anomaly, Frequency Anomaly
TXN00145	CARD1002	4026.72	Bangalore	Restaurant	PURCHASE	2026-09-22T10:17:42	55	SUSPICIOUS	Location Anomaly, Velocity Anomaly, Frequency Anomaly
TXN00142	CARD1002	5739.92	Mumbai	Walmart	PURCHASE	2026-09-22T10:17:36	15	SUSPICIOUS	Frequency Anomaly
TXN00138	CARD1002	7948.21	Mumbai	Walmart	PURCHASE	2026-09-22T10:17:28	15	SUSPICIOUS	Frequency Anomaly

Figure 20: Streamlit dashboard transaction table showing recent processed transactions.

37 Conclusion

The project implements a complete local event-driven transaction monitoring workflow using Apache Kafka and Python. The system receives continuous transaction events, validates them, applies multiple anomaly detection rules, calculates a project-defined risk score, routes records through Kafka topics, stores processed data in SQLite, generates high-risk alerts, produces JSON and log outputs, and presents the processed information through a Streamlit dashboard.

The implementation demonstrates the complete flow from transaction generation to streaming, validation, detection, classification, routing, persistence, alert generation, monitoring, and source-code version control.